

100 Days of Linux - Day 082

Title: Creating and Using Shell Variables

Today's Goal

Learn how to create, view and use your own shell variables in Linux.

Why This Matters

Shell variables make commands reusable and are the foundation of Bash scripting, automation and DevOps workflows.

Commands of the Day

Syntax:

```
NAME=Vishal
echo $NAME
AGE=26
echo $AGE
```

Examples:

```
CITY=Bangalore
echo $CITY
```

Beginner Lesson

A shell variable stores a value in memory while your terminal session is running. You create a variable using `NAME=value` without spaces around the equals sign. Use the dollar sign (\$) before the variable name to display its value.

Practice Questions

1. Create a variable named `NAME` with your name.
2. Display the value of `NAME` using `echo`.
3. Create a variable named `CITY` and display it.
4. Create variables `COURSE` and `DAY`, then display both in one command.
5. Use `echo` to print: `Welcome, $NAME to Linux!`

Hints

Do not put spaces around '='. Use `echo` followed by `$VARIABLE` to display a value.

Mini Challenge

Create variables for your name, favourite programming language and current city, then print them in a single sentence.

Common Mistakes

Adding spaces around '=', forgetting the '\$' when displaying a variable, or using invalid variable names.

Did You Know?

Variables created in a terminal normally exist only for that terminal session unless exported or saved in configuration files.

Pro Tip

Use meaningful variable names such as `PROJECT_NAME` instead of single letters.

Answer Key (Instructor Only)

- ```
Q1: NAME=YourName
Q2: echo $NAME
Q3: CITY=YourCity then echo $CITY
Q4: COURSE=Linux; DAY=82; echo $COURSE $DAY
Q5: echo "Welcome, $NAME to Linux!"
```

### Homework

Create five different shell variables and print all of them using a single `echo` command.

**Next Day Preview**

Tomorrow you'll learn how to export variables so child processes can use them.